

he Verxlite Master Blueprint

1. Brand Identity & Positioning

You need to define what Verxlite *feels* like so the UI, copy, and code architecture all align.

- The Name: Verxlite (Velocity + XR/AI + Lite)
 - The Vibe: Fast, weightless, frictionless. Heavy, manual CRM/email work is "Verx." Your AI making it effortless is "Lite."
 - The Positioning: "The Universal AI Agent for Sales & Ops."
 - The Enemy: Copy-pasting data from Gmail into Salesforce/HubSpot. Forgetting to follow up. Manual logging.
 - The Promise: "You close the deals. Verxlite handles the data."
 - The Voice: Direct, technical but accessible, confident. No buzzwords, just action. (e.g., "Meeting ended. CRM updated. Follow-up queued.")
-

2. The Core Workflow (MVP Scope)

Do not build a platform. Build a scalpel. The MVP is exactly one workflow that solves the most expensive problem in business: sales pipeline leakage.

The "Post-Meeting Velocity" Workflow:

1. Trigger: A calendar event ends (Google Calendar).
 2. Fetch: Verxlite pulls the transcript/recording (if available) or the calendar context (attendees, subject).
 3. Synthesize: LLM generates a structured summary, extracts action items, and identifies the deal stage.
 4. Execute:
 - Creates a Contact/Deal note in the CRM (HubSpot/Salesforce).
 - Drafts a personalized follow-up email in the user's drafts folder.
 - Creates a CRM task for the rep (e.g., "Send proposal by Friday").
-

3. Architecture & Tech Stack

To guarantee 100% delivery and a "wow" factor, use a Hybrid Orchestrator Pattern: Deterministic code handles routing and API calls; LLM handles reasoning.

The Stack

- Backend: Python (FastAPI) — Best LLM ecosystem.
- Agent Framework: LangGraph — Perfect for defining stateful, multi-step workflows where steps are deterministic APIs or LLM reasoning nodes.
- Database: PostgreSQL (Supabase) — Stores tenants, OAuth tokens, workflow run states.
- Queue: Redis + RQ or Celery — For background processing (meetings end, trigger runs async).
- LLM: Anthropic Claude 3.5 Sonnet (Best in class for structured data extraction/tool use) with OpenAI GPT-4o as fallback.
- Frontend: Next.js (Typekit, Tailwind, Shadcn UI).

The Data Model

- `Tenant`: Company workspace.
 - `User`: The rep/manager.
 - `Connection`: Encrypted OAuth tokens (Google, CRM).
 - `WorkflowRun`: The execution log (Status, Steps, Token cost, Duration).
 - `Artifact`: The outputs (Link to CRM note, Link to Draft email).
-

4. Step-by-Step Development Plan (100 Hours)

Phase 1: Foundation & Auth (15 Hours)

- Setup: Provision Supabase, Vercel, GitHub repo.
- DB Schema: Migrate `Tenant`, `User`, `Connection`, `WorkflowRun` tables.
- Auth: Implement Clerk or Supabase Auth.
- OAuth - Google: Request `gmail.modify`, `calendar.readonly`, `drive.readonly`. Save encrypted tokens.

- OAuth - CRM: Implement HubSpot OAuth (easiest for MVP). Save encrypted tokens.
- *Checkpoint*: User can log in, connect Google, and connect HubSpot.

Phase 2: The Trigger & Fetchers (15 Hours)

- Google Webhooks: Register push notifications for Google Calendar so Verxlite knows instantly when a meeting ends.
- Fetcher - Calendar: Write function `get_event_details(event_id)`.
- Fetcher - Email: Write function `get_sent_emails_thread(contact_email)`. (Context on past comms).
- Fetcher - CRM: Write function `get_crm_deal(contact_email)`.
- *Checkpoint*: When a meeting ends, Verxlite automatically aggregates Calendar + Email + CRM context into a single JSON payload.

Phase 3: The Agent Logic (LangGraph) (25 Hours)

This is the brain. Do not put this in one giant prompt. Break it into nodes.

- Node 1: Router. Deterministic check. Does this meeting have a CRM contact? Yes -> Proceed. No -> Flag for manual review.
- Node 2: Reasoner. LLM call. Prompt: *"Analyze this meeting context. Output JSON: {summary, action_items[], next_steps, sentiment}"*.
- Node 3: CRM Updater. Tool call. LLM formats the API payload, deterministic code executes the HubSpot API call to create a Note on the Deal.
- Node 4: Email Drafter. Tool call. LLM drafts the follow-up, deterministic code pushes it to Gmail Drafts via API.
- Node 5: Task Creator. Tool call. LLM defines the task, deterministic code pushes to HubSpot Tasks.
- *Checkpoint*: The agent runs end-to-end in the background. Test with Postman/cURL.

Phase 4: Observability (The "100% Delivery" Insurance) (15 Hours)

If you cannot see the agent thinking, you cannot fix it. This is mandatory.

- Integrate Langfuse or Braintrust.
- Wrap every LLM call and Tool call with a trace.

- Log: Input tokens, Output tokens, Latency, Tool payload, Success/Failure.
- *Checkpoint*: Go to Langfuse dashboard and see the exact path the agent took, where it slowed down, and what it decided.

Phase 5: Frontend & UX (20 Hours)

Verxlite must feel "Lite" (fast, clean, invisible).

- Dashboard: Simple table of recent `WorkflowRuns`.
- Run Detail View:
 - Show the Meeting Title.
 - Show the AI Summary.
 - Provide deep links: "View in HubSpot", "View in Gmail Drafts".
- Manual Trigger: A button to "Sync Past 7 Days" of meetings for initial onboarding.
- *Checkpoint*: The UI is functional and directly links out to the places reps already live (CRM, Gmail).

Phase 6: Hardening & Polish (10 Hours)

- Idempotency: If the same meeting webhook fires twice, Verxlite must NOT create duplicate CRM notes. Check `event_id` against past runs.
- Error Handling: If the LLM fails, put the run in "Needs Review" state, don't crash the queue.
- Styling: Apply the Verxlite brand (Black, stark white, sharp neon blue or green accents—emphasizing speed/lightness).

5. Developer Rules for Guaranteed Success

1. No "Agentic" Loops in MVP: Do not let the LLM loop infinitely trying to solve a problem. Force it to output a structured JSON or escalate to the user. Infinite loops = infinite API costs and broken UX.
2. Strict JSON Mode: When asking the LLM to extract data or draft emails, enforce JSON output schemas (e.g., `{"subject": "...", "body": "..."}`). It makes parsing deterministic.

3. Never Send Without Approval (MVP): The agent creates *Drafts* in Gmail, it does not hit *Send*. This builds trust. You can add "Auto-Send" as a premium feature later.
 4. The 2-Second Rule: The webhook response to Google must return `200 OK` in under 2 seconds. Do all the heavy lifting (LLM calls, CRM updates) in the background queue.
 5. Mock the APIs First: Before writing the HubSpot integration, write a mock HubSpot API that returns fake data. Get the agent working end-to-end with the mock, *then* swap in the real API.
-

6. The Verxlite Landing Page Copy (For when you launch)

Hero Section:

Stop Logging. Start Closing.

Verxlite is the universal AI agent that turns your meetings into CRM updates, follow-ups, and tasks. Instantly. Zero data entry.

How it Works:

1. You finish a meeting. Verxlite detects it automatically.
2. Verxlite listens & learns. AI extracts action items, deal details, and next steps.
3. Your CRM is updated. Notes logged, tasks created, follow-up drafted. You just click 'Send'.

The Footer Tagline:

Verxlite — The weight of manual work, lifted.

Build exactly what is written here. Do not add a settings page yet. Do not add custom workflow builders yet. Ship the "Post-Meeting Velocity" workflow flawlessly. That is how Verxlite wins.